

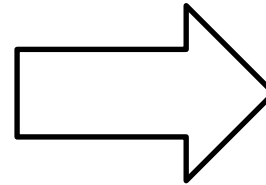
The failure of PKI
→ *Certificate Transparency*

*(with insights on hash based data
structures, specifically Merkle trees)*

*a VERY SERIOUS real world problem: fake
certificates*

Web security pillar: Certificate Authorities **ARE** trusted!

UNIVERSETRUST

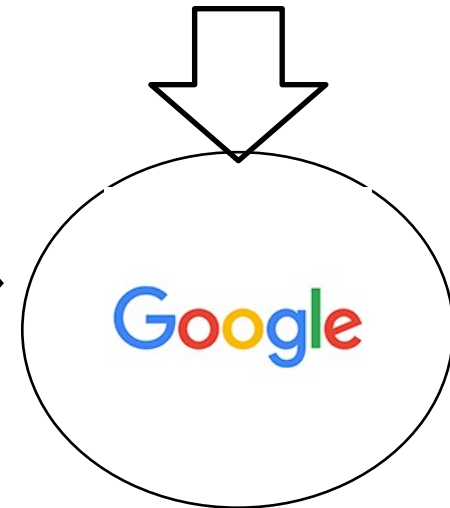


LUNATRUST



<https://www.google.it>

CERTIFICATE(I am google)_{LUNATRUST}



Fact: trusted CA assumption at stake

Fake SSL certificates threats, say



by George Leopold

Published: 09 Jan 2015

Home News Anti-Phishing

Fake SSL certificates

Netcraft has found dozens of fake SSL certificates. Some of these certificates may be used by attackers to impersonate legitimate websites and steal customers' sensitive information, such as usernames, passwords, and credit card details.

The fake certificates bear the Netcraft logo, but are not signed by Netcraft.



Online Security Blog

The latest news and insights from Google on security and safety on the Internet

Enhancing digital certificate security

Posted: Thursday, January 3, 2013

g+

183

Tweet

300

Like

→ Google's VALID fake Certificates mistakenly (?) issued

→ by TurkTrust (2012), ANSSI France (2013), etc

→ Smaller CAs: compromised

⇒ Holland: Dgnotar

⇒ Malaysia: DigiCert sdn. Bhd.

⇒ etc

Engineer

ected and blocked an unauthorized digital certificate for the "*.google.com" domain. We found the certificate was issued by an [intermediate certificate authority](#) (ICA) from a Turkish certificate authority. Intermediate CA certificates carry the full authority of the root CA, and anyone who has one can use it to create a certificate for any website they wish to.

certificate revocation metadata on December 25 to block that intermediate CA, and other browser vendors. TURKTRUST told us that based on our information, they had not received any notification from the Turkish government.

TLS Proxies: Friend or Foe?

Mark O'Neill, Scott Ruoti, Kent Seamons, Daniel Zappala
Brigham Young University
Department of Computer Science
Provo, UT 84602

Giuseppe Bianchi

A serious problem

→ Fake VALID certificates

- ⇒ Various small/moderate CA «releasing» valid certificates for major sites
 - Asked by governments? Mistakenly... lost?
 - Many cases:
 - google fake certificates issued by
 - » Turkey's Turktrust, December 2012
 - » ANSSI in France on December 2013
 - » etc

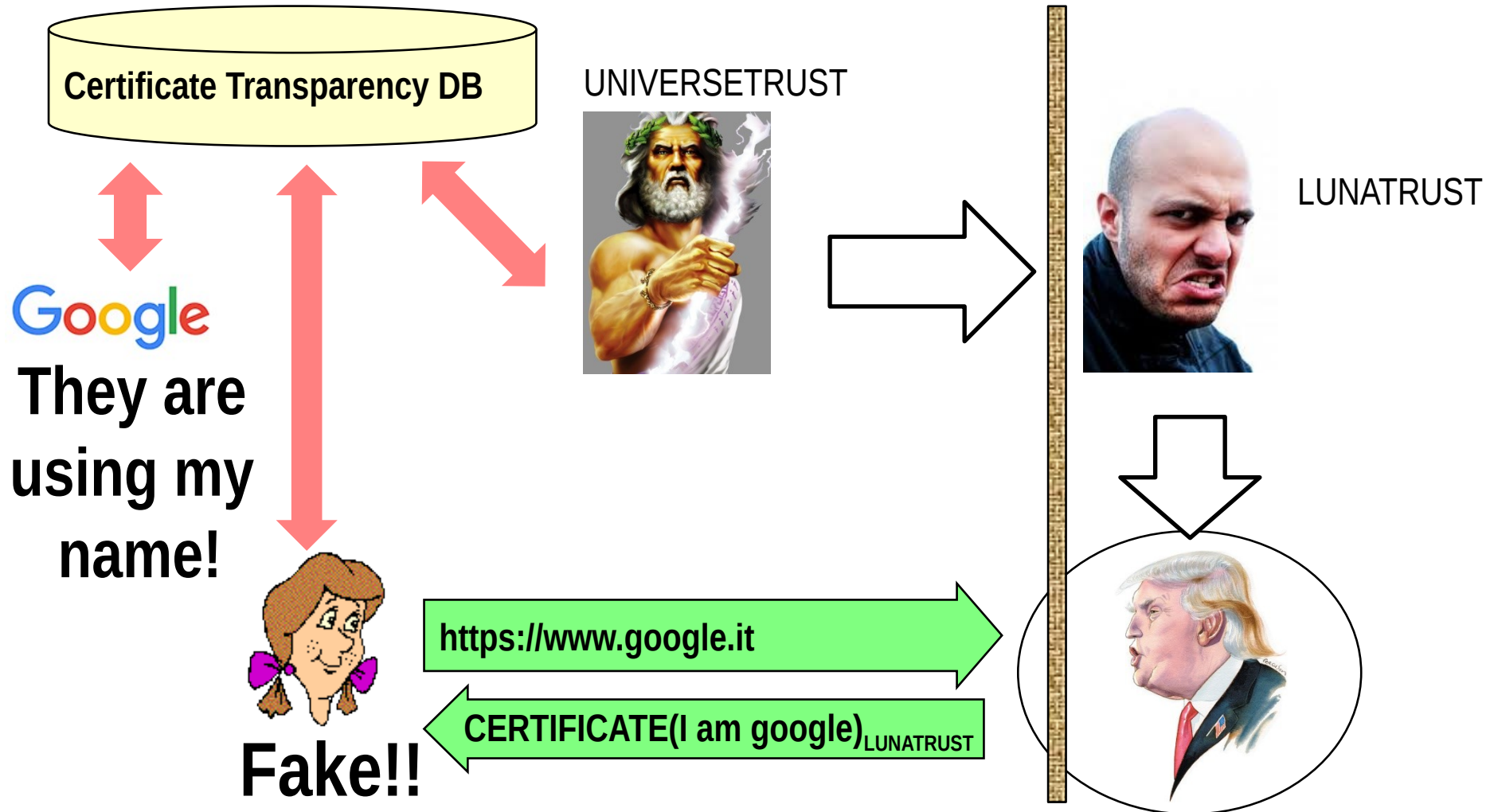
→ Compromised CAs

- ⇒ Holland: DGnotar
- ⇒ Malaysia: DigiCert sdn. Bhd.
- ⇒ etc

**With powerful threats (governments), and many players which can «make mistakes»
the security of the PKI model is getting weaker and weaker**

How to cope with malicious CAs?

Idea: gigantic worldwide DB which anyone can check!



*How to implement such gigantic Database of
certificates?*

A parenthesis: Merkle Trees

How to secure a file

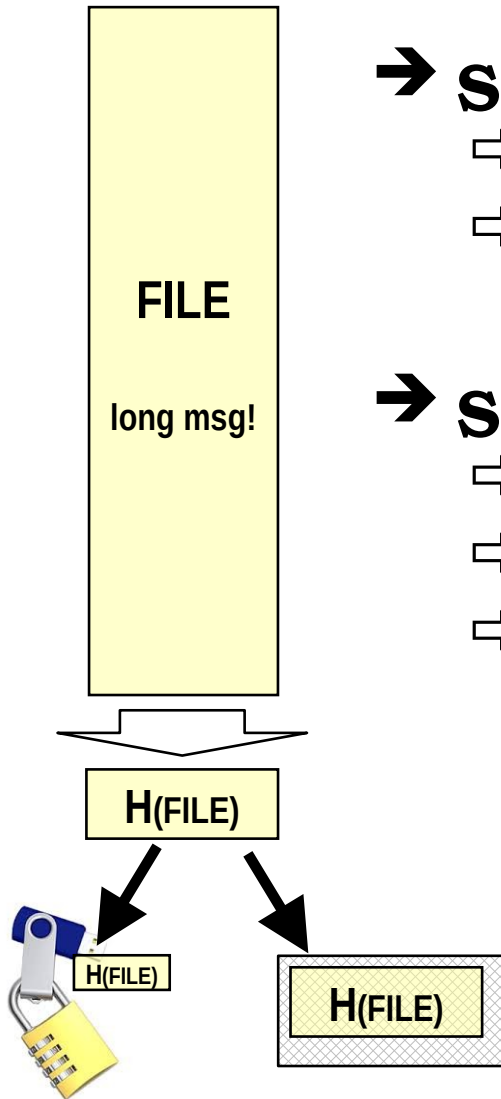
(when you don't have a large secure storage available)

→ Step 1: make “fingerprint” of the file

- ⇒ Via a cryptographic hash function
- ⇒ large file → small fingerprint!

→ Step 2: “secure” the fingerprint!

- ⇒ By copying it in a secure storage
- ⇒ ... or ...
- ⇒ By digitally signing the hash fingerprint

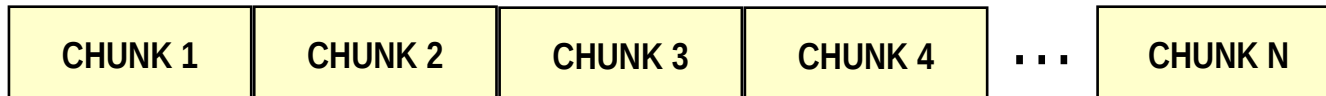


What about «chunked» messages?



→ Peer-to-peer delivery

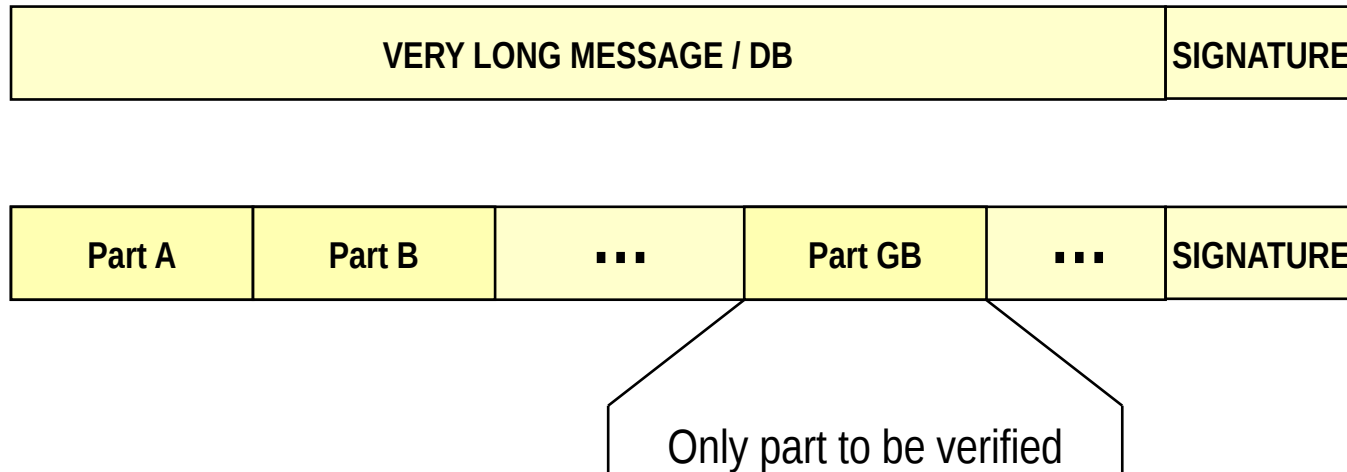
- ⇒ Message divided into independent chunks
- ⇒ Frequently received out of order, and from distinct peers



→ Signature verification: needs to wait until **COMPLETE** message reconstruction

- ⇒ But what about fake injected chunks?

What about «partial» verification?



**Signature verification:
need to retrieve the whole database??**

Per-chunk signatures?

CHUNK 1	SIG1
---------	------

CHUNK 2	SIG2
---------	------

CHUNK N	SIG-N
---------	-------

→ OK but...

→ **Computational overhead**

⇒ Signature = asymmetric crypto = $O(1000+)$ x hash computation cost

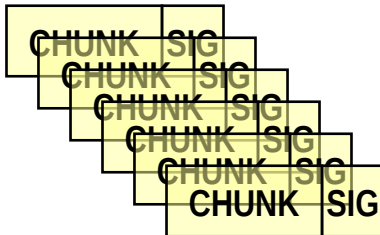
→ **Storage overhead**

⇒ RSA2048 = 256 bytes

⇒ Bitcoin's ECDSA256 = $2 \times 256 \text{ bit} = 64 \text{ bytes}$

→ still quite a lot if you need MANY of them (small chunks)

→ **No more integrity check for whole file (strip-type attacks)**



Per-chunk signatures?

CHUNK 1	SIG1
---------	------

CHUNK 2	SIG2
---------	------

CHUNK N	SIG-N
---------	-------

→ OK but...

→ **Computational overhead**

⇒ Signature = asymmetric crypto = $o(1000+)$ x hash computation cost

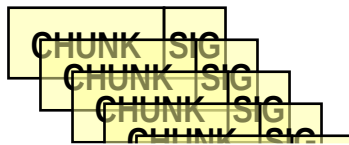
→ **Storage overhead**

⇒ RSA2048 = 256 bytes

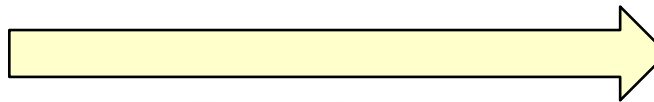
⇒ Bitcoin's ECDSA256 = $2 \times 256 \text{ bit} = 64 \text{ bytes}$

→ still quite a lot if you need MANY of them (small chunks)

→ **No more integrity check for whole file (strip-type attacks)**

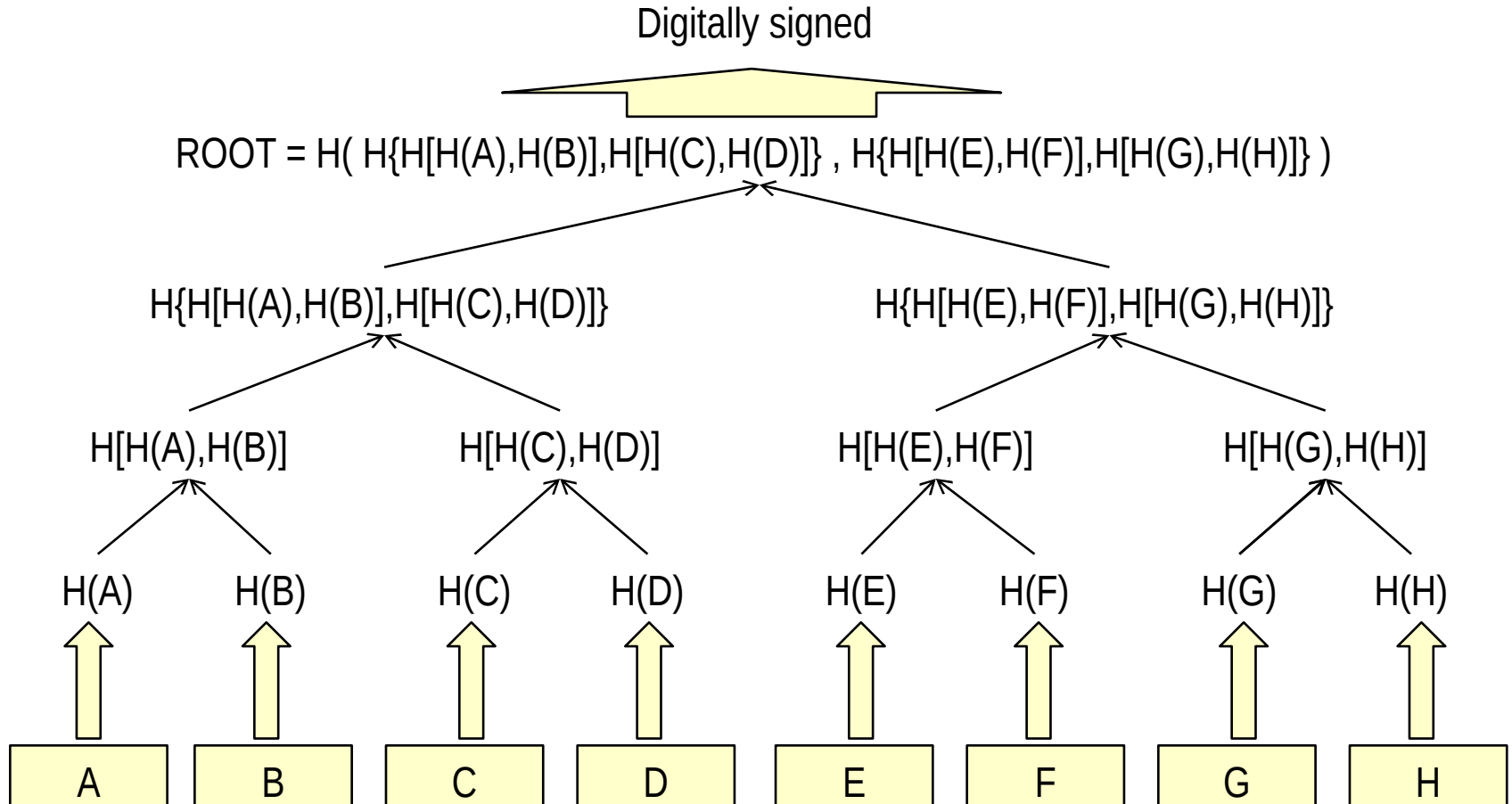


**Can we find
a better solution?**

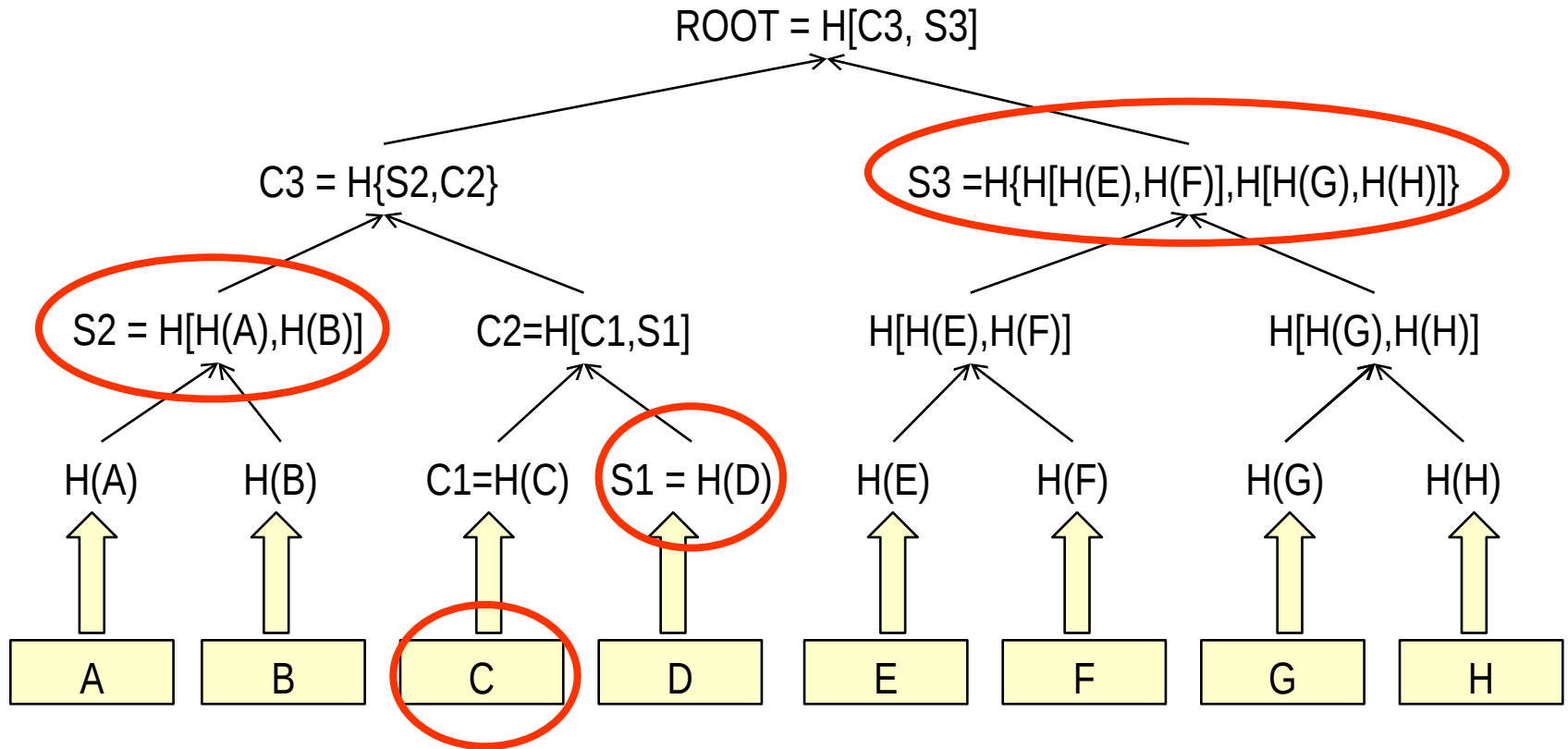


*Ops... did I
receive all?*

Merkle's idea: Hash tree

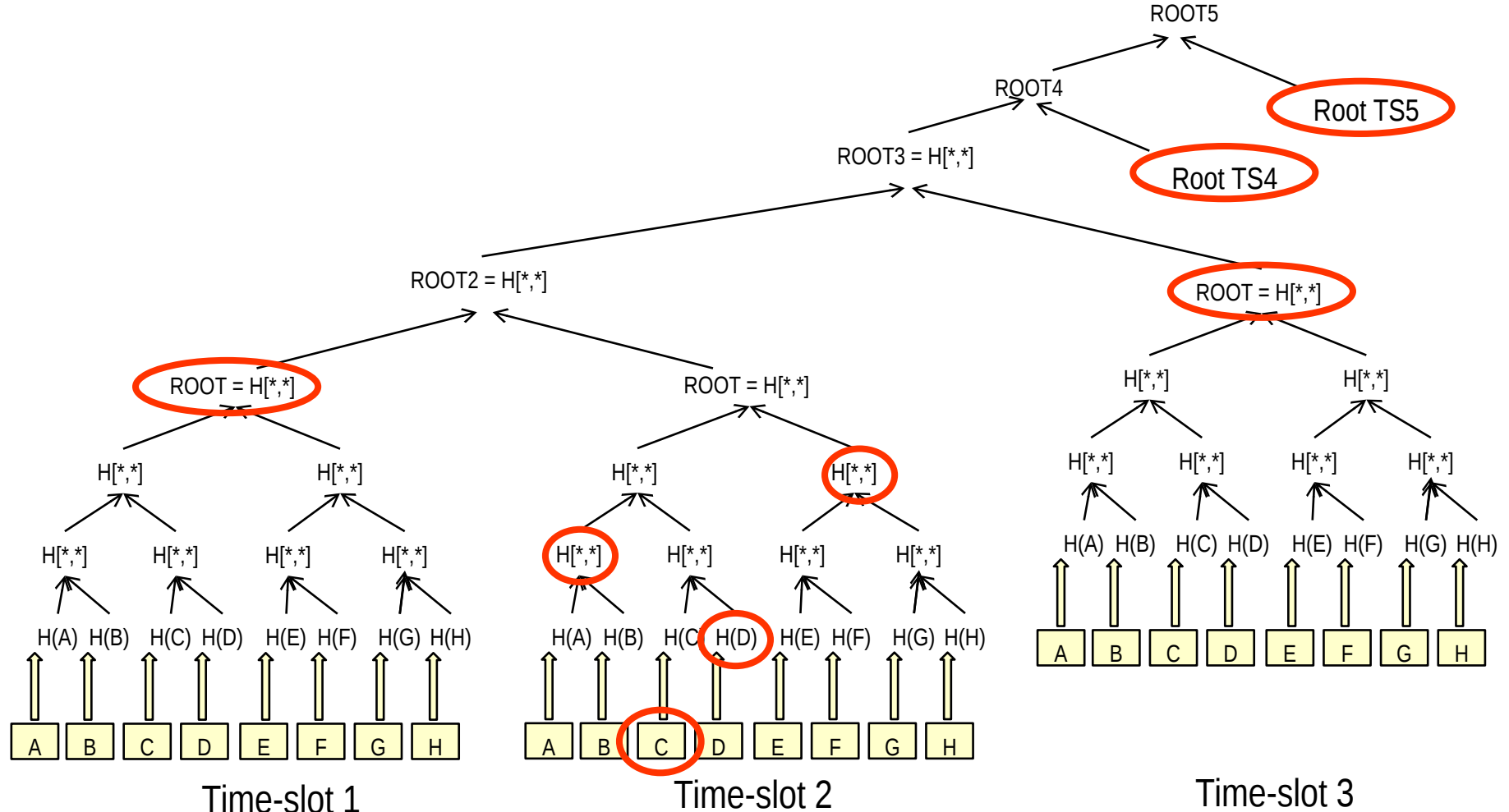


Single chunk verification: use “siblings”



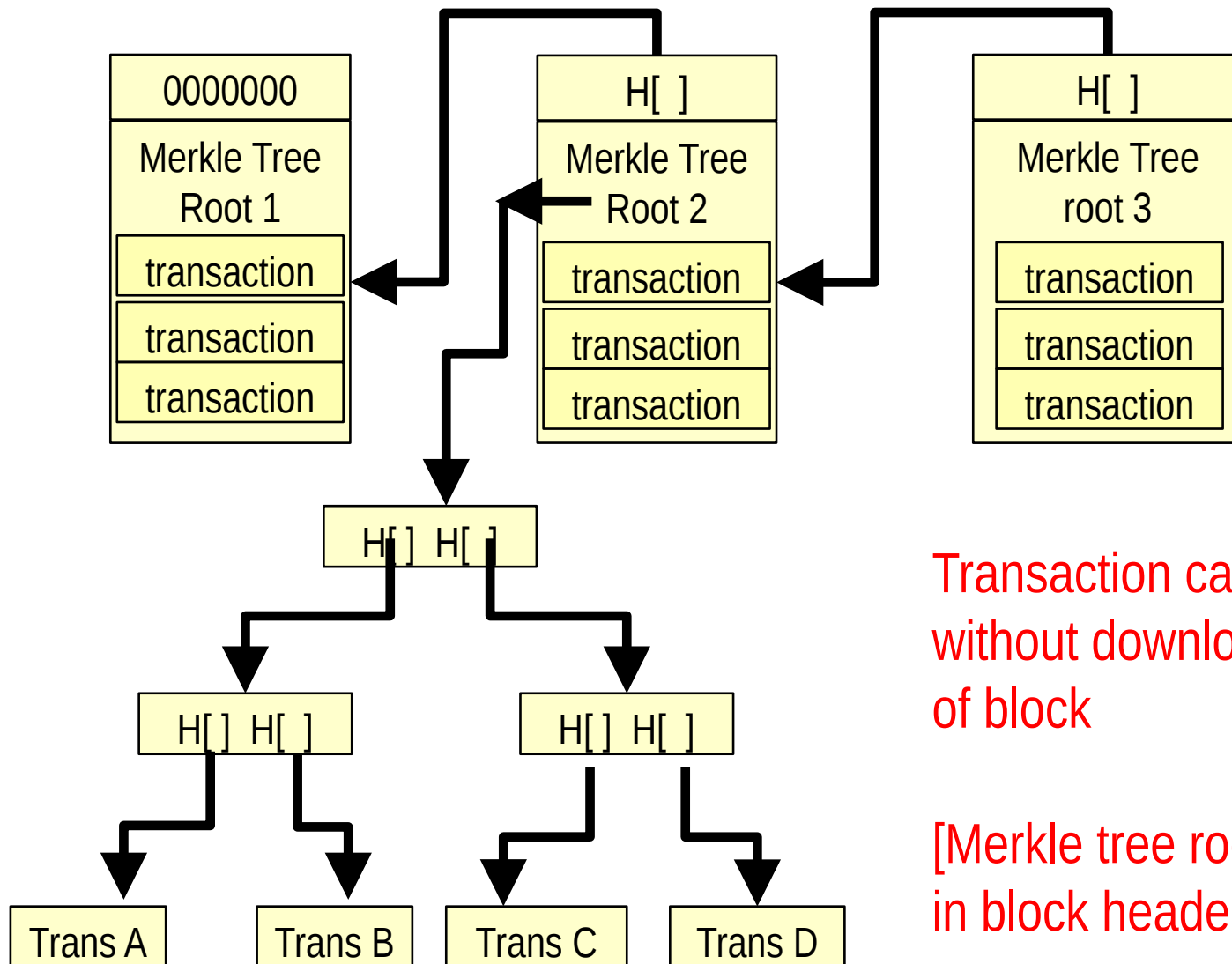
Log(N) siblings needed

Extending merkle's trees w. time



VALIDATION: Siblings for «your» tree + root of previous timeslot + roots of next timeslots

Merkle trees and blockchains



Transaction can be verified
without downloading content
of block

[Merkle tree root stored
in block header]

Merkle Trees: Applications

→ **Efficient storage in blockchains**

⇒ Bitcoin, Ethereum, etc

⇒ Used in many parts of many blockchains!

→ **Google's Certificate transparency**

→ **File Chunk validation**

→ **And many more**

⇒ One-time signatures

⇒ Node authentication

⇒ Memory & storage authentication

⇒ Signature spreading

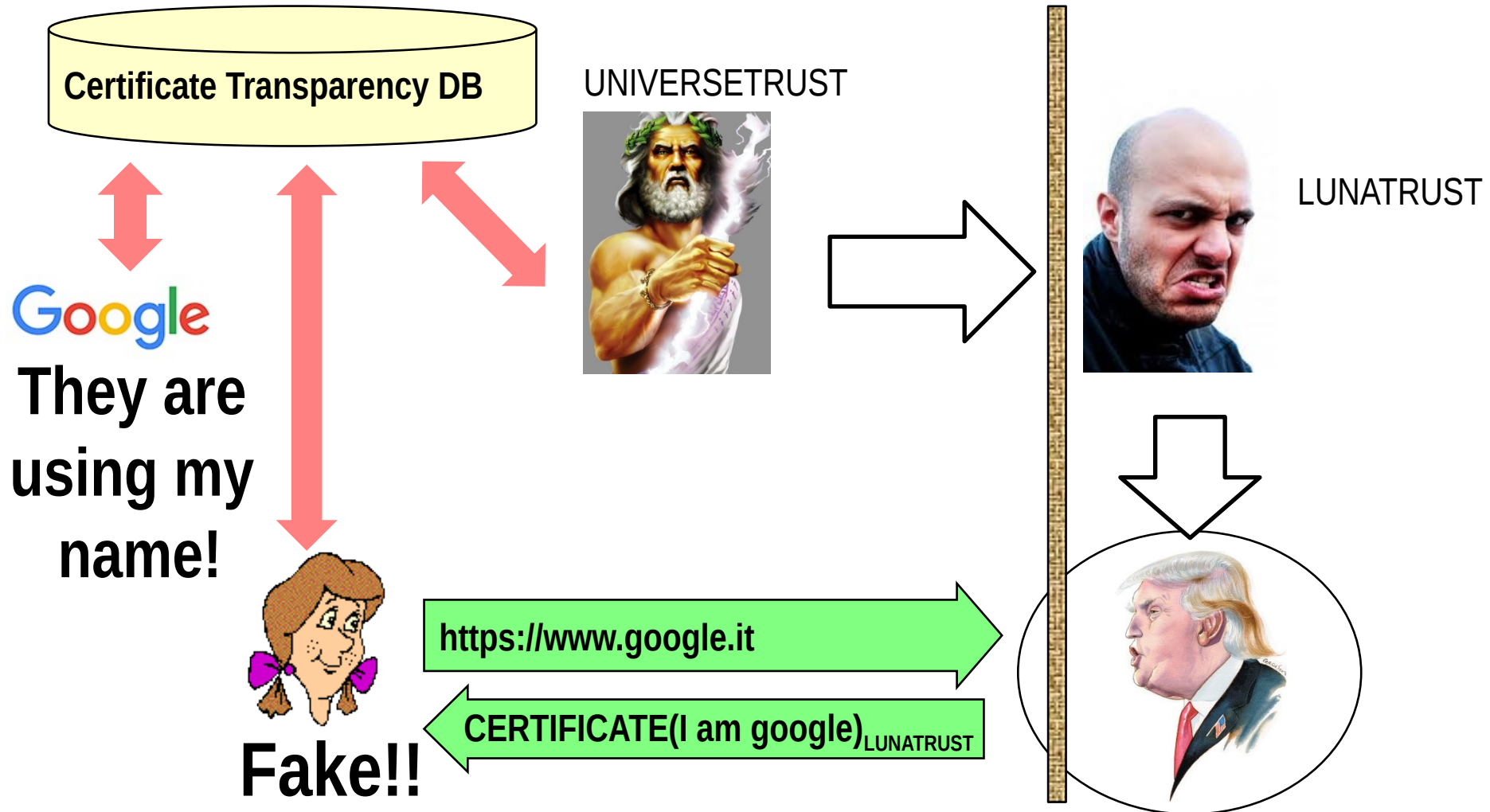
⇒ Etc etc ...

Merkle Trees' application:

Certificate Transparency

How to cope with malicious CAs?

Idea: gigantic worldwide DB which anyone can check!



Why certificate transparency?

- ➔ **Make it almost impossible for a CA to issue a SSL certificate for a domain without the certificate being visible to the owner of that domain.**
- ➔ **Idea: put all certificates in a gigantic public list, that everyone can access and consult**
- ➔ **Further idea: make sure that we all see the “same” list!!**
 - ⇒ New threat: “split world”: the public list you see is different (and crafted!) from what the rest of the world sees

Certificate Transparency in a nutshell

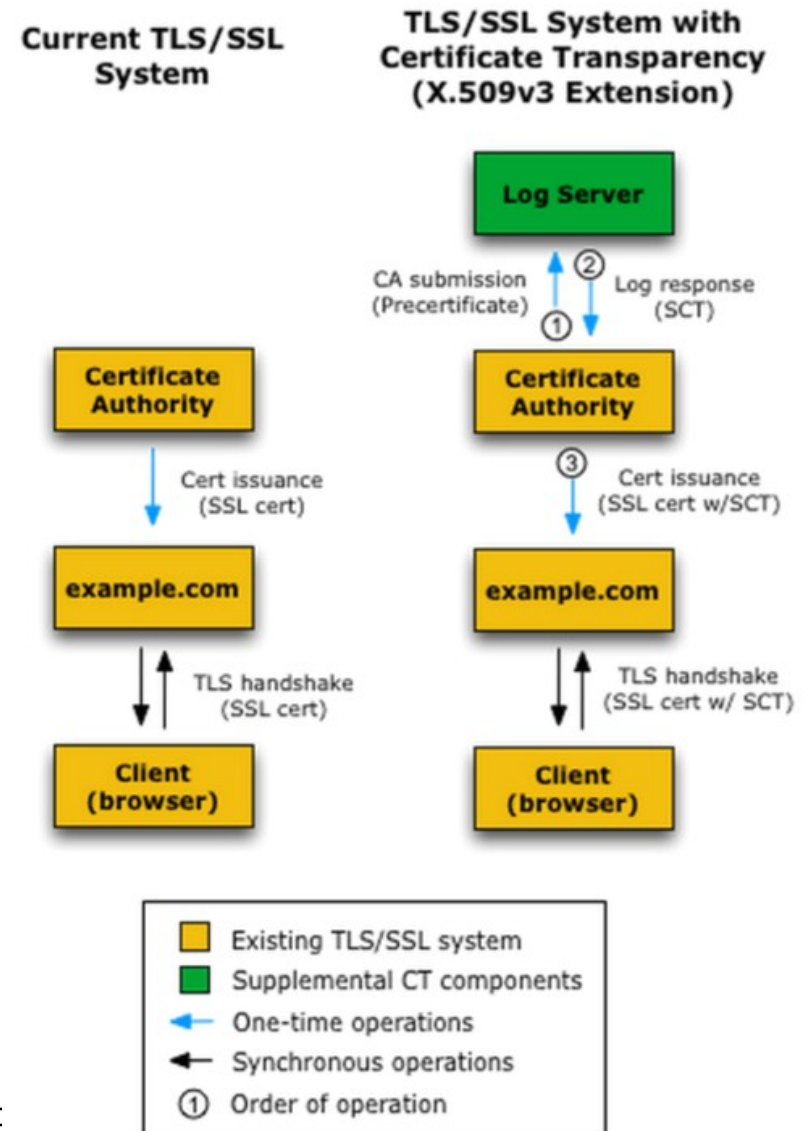
→ **When issuing certificate: add it to the log server**

⇒ Returns a Signed Certificate Timestamp (SCT)

→ **During TLS handshake:**

⇒ Check certificate validity (as usual)

⇒ Follow up with an **additional** Extended Validation (EV) by verifying that the certificate is in the public list



Certificate Transparency

- **Google's initiative**
 - ⇒ started 2013
- **First stated in experimental RFC 6962**
 - ⇒ July 2013
- **Endorsed and supported by IETF WG «trans»**
 - ⇒ Goal: standardize CT
 - ⇒ RFC 6962bis released in December 2014
- **Integrated in major browsers**
 - ⇒ Chrome 1+, IE7+, Firefox 3+, Safari 3.2+, Opera 9.5+
 - ⇒ Although not yet consistently used by all web sites
- **Supported by major sites**
 - Paypal, CertSign, etc
- **Multiple pilot logs**
 - Google, Cloudflare, DigiCert, etc
 - <https://certificate.transparency.dev/logs/>
- **<https://crt.sh/>**

CT: how it works?

→ Uses Merkle trees!

⇒ Very scalable, $\log(n)$ verification complexity

→ A special type (sometimes called chron tree)

⇒ «append only» Merkle tree

⇒ Timestamped entries

→ Cannot retroactively insert a certificate

Following figures: source <http://www.certificate-transparency.org/log-proofs-work>

CT: how it works?

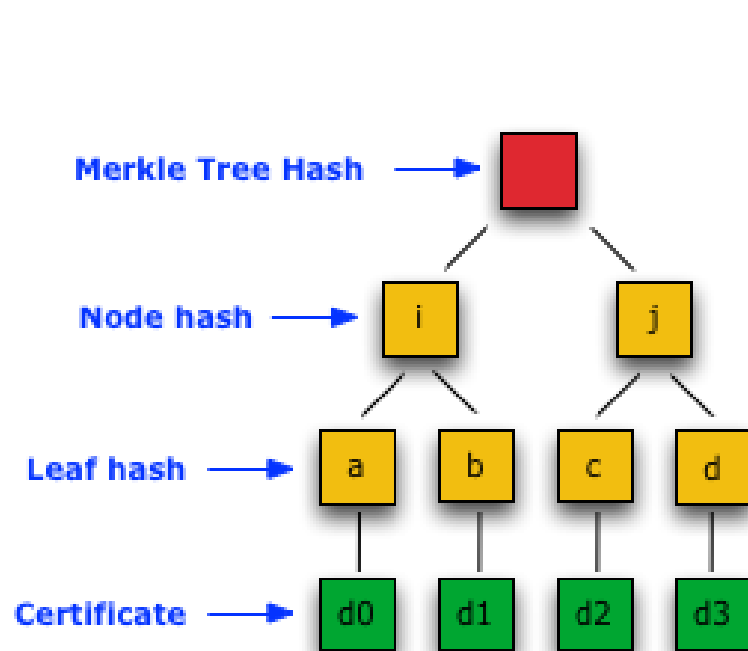


Figure 1

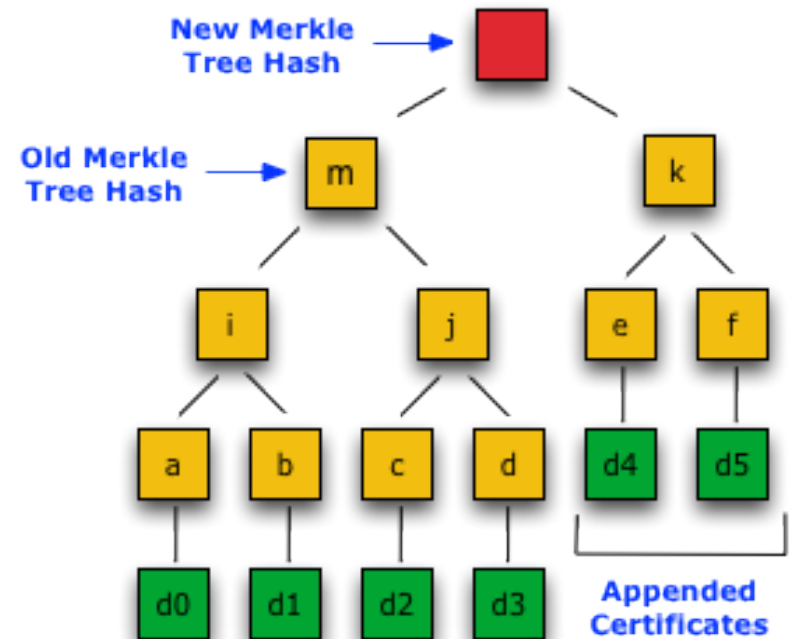


Figure 2

Every X hours, tree updated (currently X=24h)

Merkle Consistency proof

Verify that two versions of a log are consistent (i.e., the later includes everything of the earlier).
Consistency proof $\rightarrow$ no certificates have been modified or back-dated and inserted into the log.
Proof = need only root of the subtrees – includes past “root”

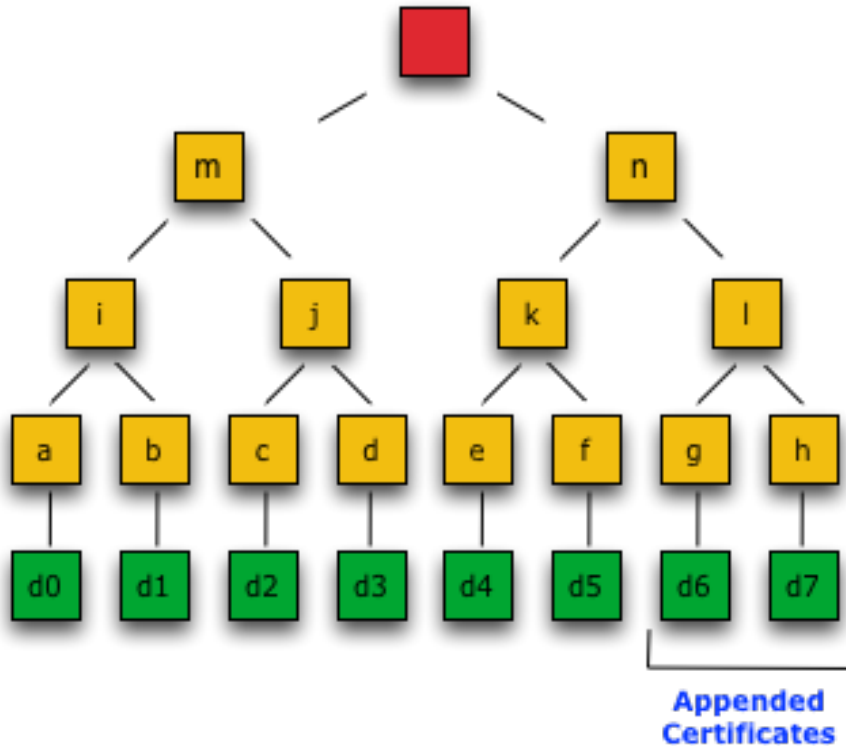


Figure 3

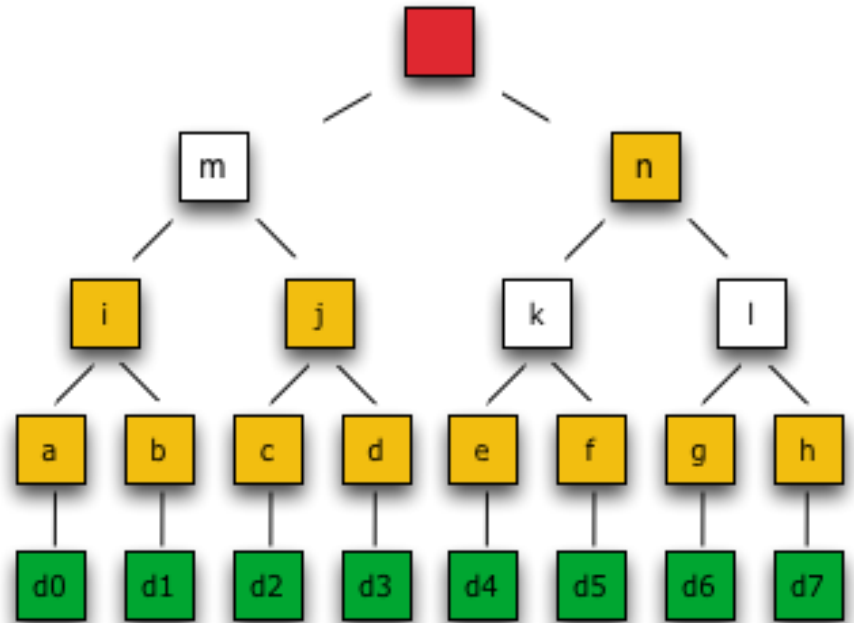


Figure 4

Merkle Audit Proof

- **Usual: via certificate + siblings**
- **Siblings obtained from log**
 - ⇒ Issue: online operation, visible to all...
 - ⇒ Remember who are the threats today...
- **Alternative: use «gossiping» protocols**
 - ⇒ Work in progress @ 2015!
 - ⇒ No time for (very nice) details...

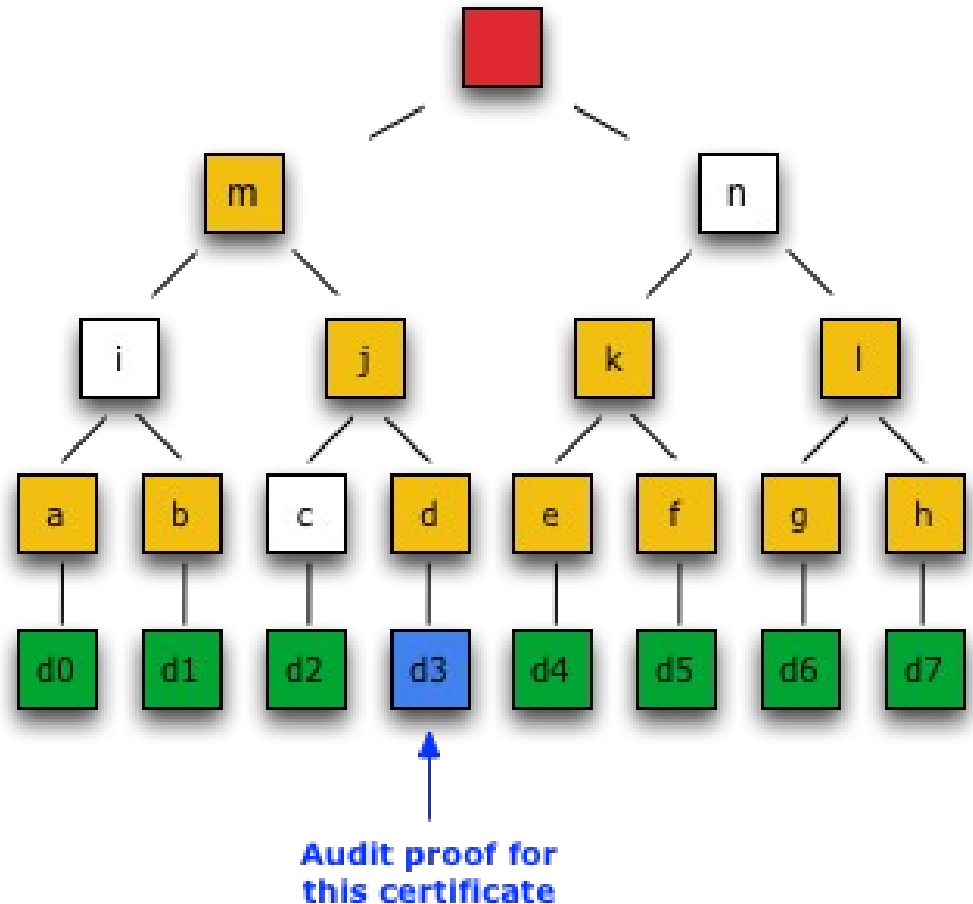
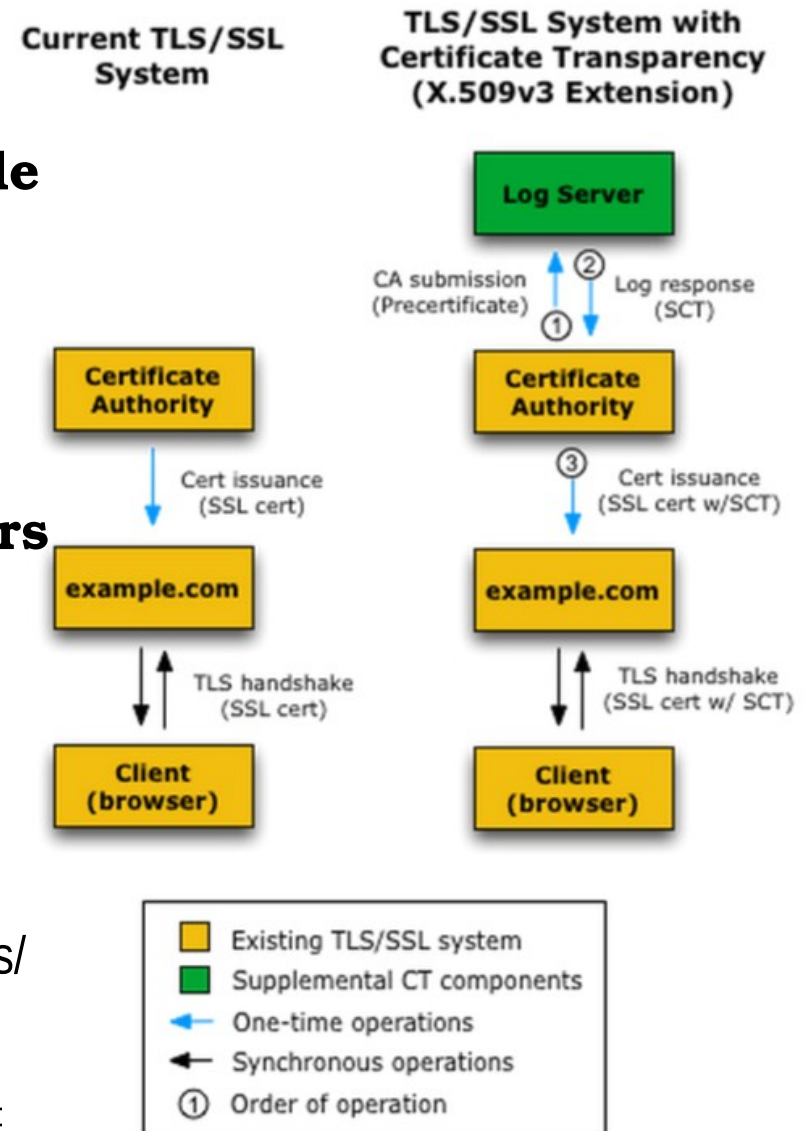


Figure 5

Certificate Transparency in the real world

- Launched in July 2013 by Google
- experimental IETF RFC 6962
 - IETF WG «trans»
 - Specified processes and protocols
- Integrated in all major browsers
- Supported by major sites
 - Paypal, CertSign, etc
- Multiple pilot logs
 - Google, Cloudflare, DigiCert, etc
 - <https://certificate.transparency.dev/logs/>



EV in the real world

PayPal, Inc. [US] <https://www.paypal.com/it/webapps/mpp/home>

La tua connessione a questo sito è privata.

Autorizzazioni **Connessione**

L'identità di PayPal, Inc. a San Jose, California US è stata verificata da Symantec Class 3 EV SSL CA - G2 e può essere controllata pubblicamente.
[Informazioni certificate](#)
[Informazioni sulla trasparenza](#)

La connessione a www.paypal.com è crittografata tramite crittografia obsoleta.

La connessione utilizza TLS 1.2.

La connessione è stata crittografata utilizzando AES_256_CBC, con SHA1 per l'autenticazione dei messaggi e RSA come meccanismo principale di scambio delle chiavi.

Informazioni sito
Non hai mai visitato questo sito prima di oggi.

[Che cosa significano?](#)

Visualizzatore timestamp certificato firmato

1: Integrato, Verificato

Stato di convalida	Verificato
Origine	Integrato
Versione	V1
Nome log	Google 'Pilot' log
ID log	A4 B9 09 90 B4 18 58 14 87 BB 13 A2 CC 67 70 0A 3C 35 98 04 F9 1B DF B8 E3 77 CD 0E C8 0D DC 10
Emesso alle ore	giovedì 23 aprile 2015 00:28:58
Algoritmo di hash	SHA-256
Algoritmo di firma	ECDSA
Dati della firma	30 45 02 20 5B E3 52 37 1B 84 B4 48 D1 CD 8F 53 34 D5 31 22 2D 46 C1 91 B2 86 6A 77 3E 37 DF FD CC 9D 78 A5 02 21 00 DE 96 80 20 B6 82 09 1E 8E 4A 9F C7 EE 3B 35 47 82 31 B1 D0 B1 63 F7 7D 52 1C C5 C5 41 5E BF 1A

Chiudi

Common misconception: Cert Trans is a Blockchain?

→ It definitely looks like

⇒ Identical (!) architecture as Bitcoin's ledger

→ It could be distributed as well

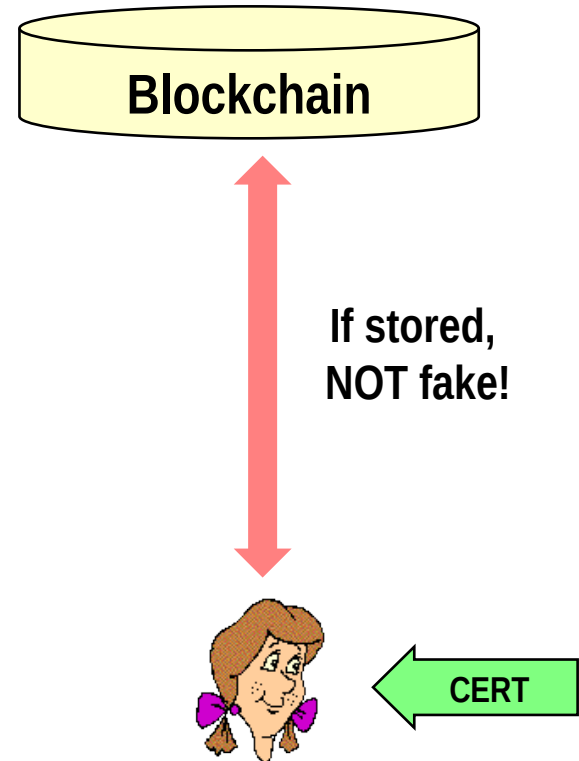
⇒ Instead of different logs, make it a same «consensus» log

→ But it is NOT a Blockchain

⇒ It does NOT necessarily contain only VALID certificates!

→ As a Blockchain mandates; also Google being crystal clear on this!

→



Cert Trans → is a Blockchain?

- **It definitely looks like**
 - ⇒ Identical (!) architecture as Bitcoin's ledger
- **It could be distributed as well**
 - ⇒ Instead of different logs, make it a same «consensus» log
- **But it is NOT a Blockchain**
 - ⇒ It does NOT necessarily contain only VALID certificates!
 - As a Blockchain mandates; also Google being crystal clear on this!
- **Security comes from transparency**

